CPE112HDS — Module1 — Lecture1

Lecture1:

Python-OOP

(Object Oriented Programming)

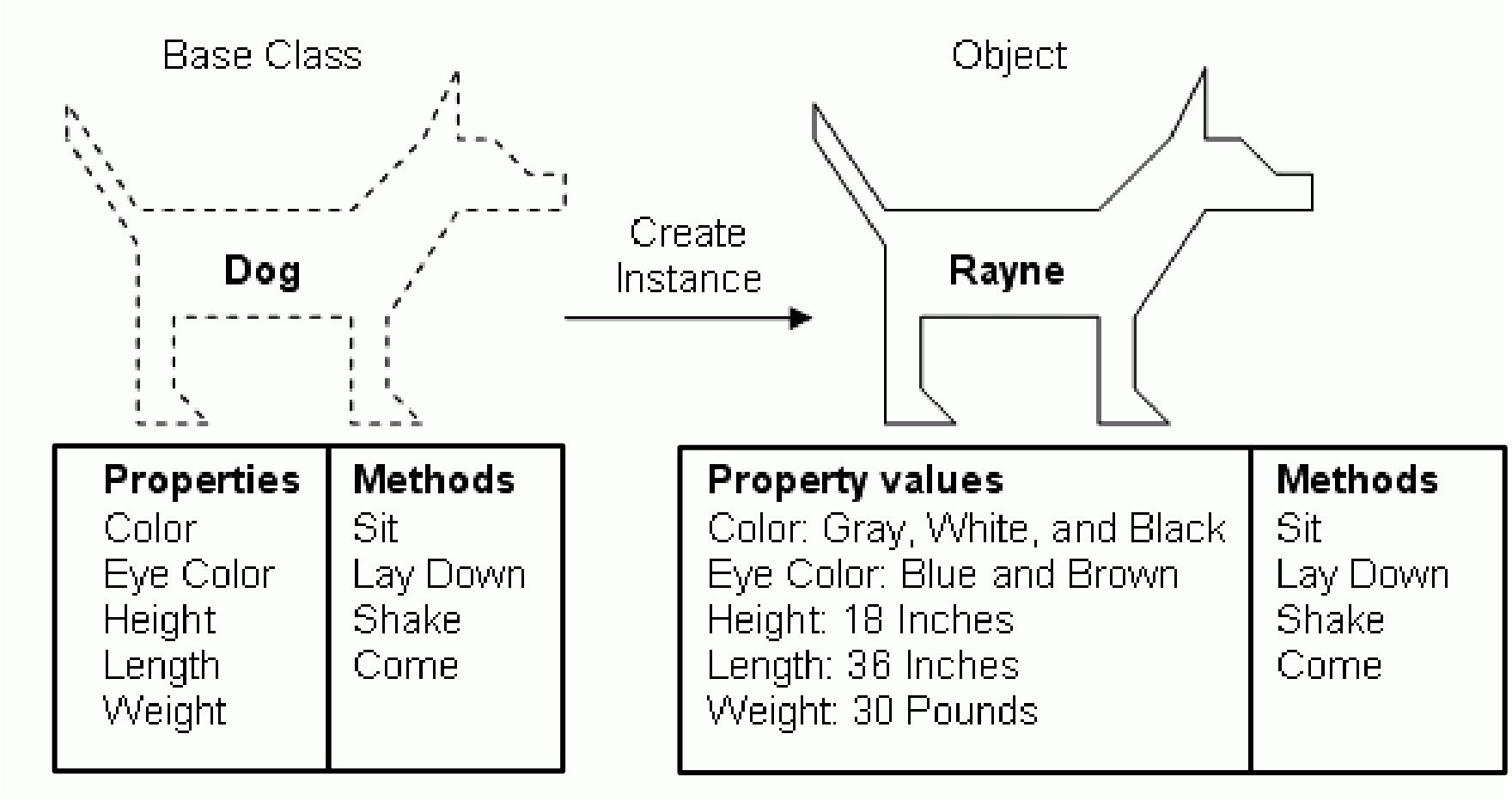
Dr. Prapong Prechaprapranwong

Computer Engineering

King Mongkut's University of Technology Thonburi

What is an **Object**?

Each <u>object</u> is an <u>instance</u> of a <u>class</u>



Why OOP

Object Oriented Programming

Robustness:

capable of handing unexpected inputs that are not explicitly defined for its application

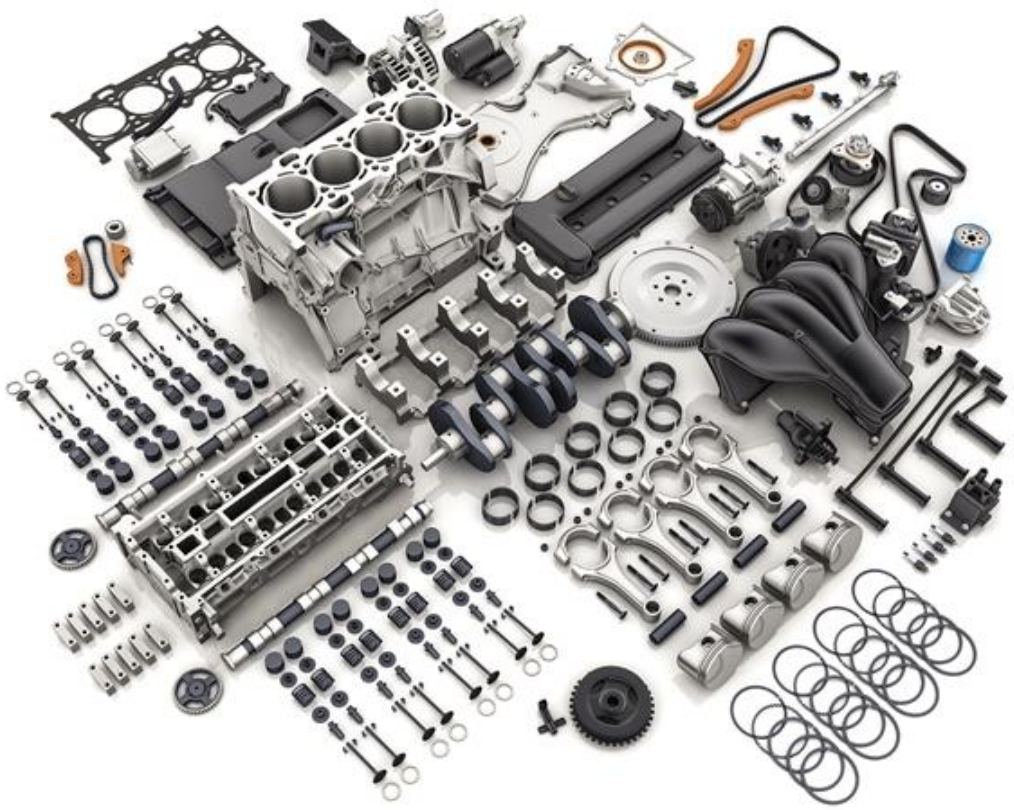
Adaptability (also called evolvability or portability): the ability of software to run with minimal change on different hardware and OS platforms

Reusability:

the same code should be usable as a component of different system in various application

Object-Oriented Design Principles

Modularity refers to an organizing principle in which different components of a software system are divided into separate functional units.



Module is a collection of closely related functions and classes that are defined together in a single file of source code

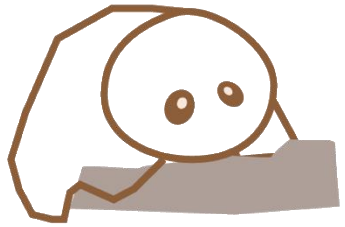
e.g. *math* module

Object-Oriented Design Principles

Abstraction

is to refine a complicated system down to its most fundamental parts

Low level of abstraction
It's a panda! Asking himself some philosophical questions about life!



Medium level of abstraction
We're not quite sure about the animal, but we recognize that it's a face and body.

High level of abstraction
Could be a great logo or design element, but the relationship with the panda has to be learned/guessed

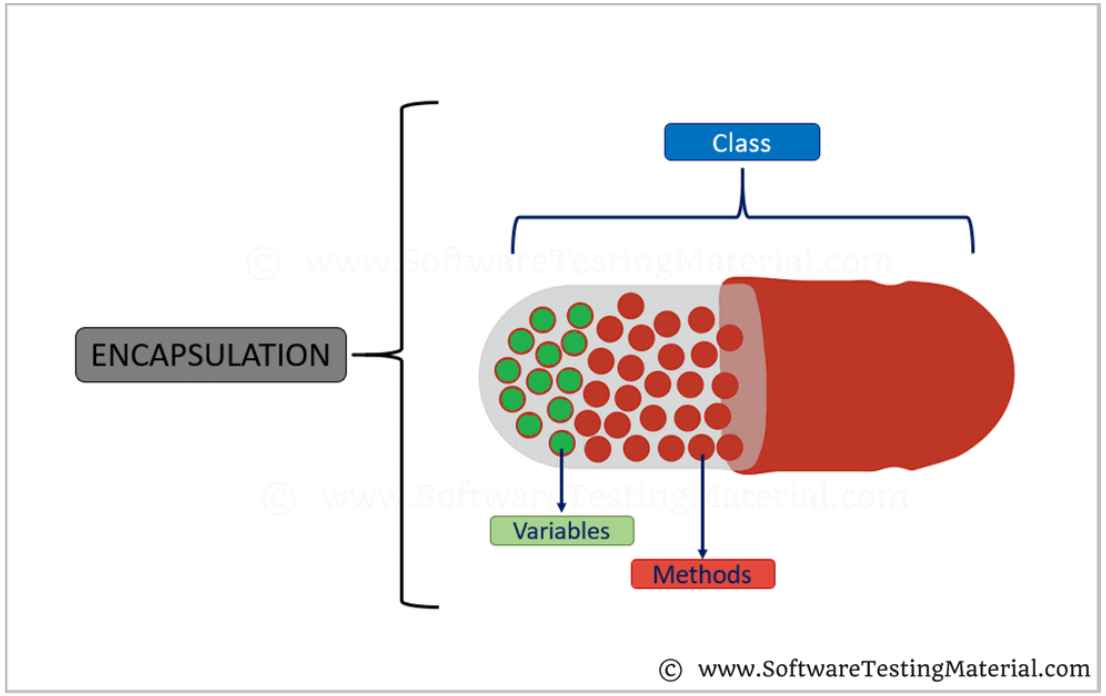
– describing the parts of a system involves naming them and explaining their functionality –

This is [Design Abstraction]

Object-Oriented Design Principles

Encapsulation different components of a software system should not reveal the internal details of their respective implementations.

One of the main advantages of encapsulation is that it gives one programmer freedom to implement the details of a component, without concern that other programmers' details



Class objects support two kinds of operations:

- Instantiation
- Attribute references.

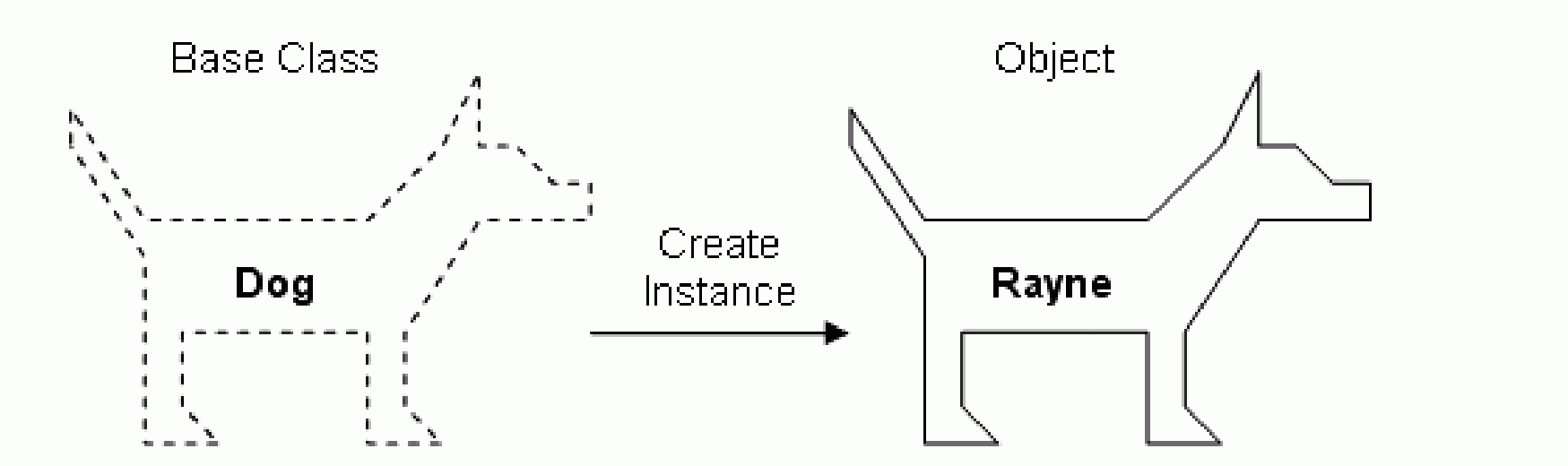
```
class ClassName:
    <statement-1>
    ...
    <statement-N>
```

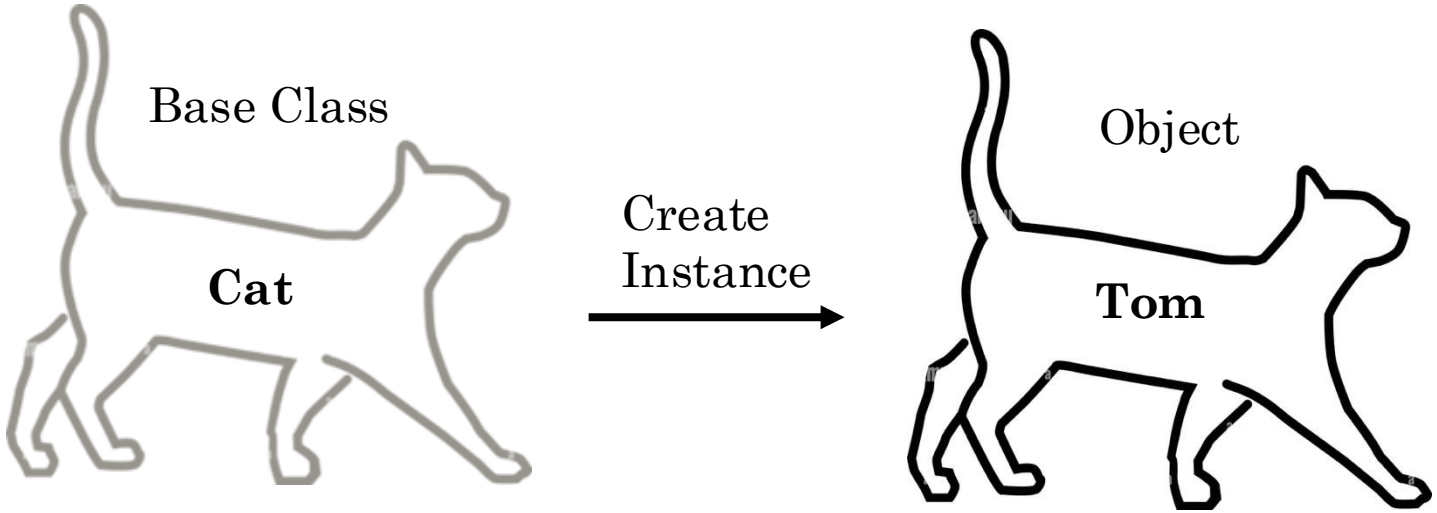
Class instantiation uses function notation. Just pretend that the class object is a function that returns a new instance of the class

Example:

```
aMan = Person()
```

```
class Person:
  '''A simple example class'''
  name = "Peter"
  def showdetail(self):
    self.citizen_id = 3701516086894
    print("name: {}".format(self.name))
    print("citizen ID: {}".format(self.citizen_id))
```


Base Class
Dog
Create
Instance
Object
Rayne


Base Class
Cat
Create
Instance
Object
Tom

Attribute references

use the standard syntax used for all attribute references in Python

e.g. obj.name

Example:

```
aMan.showdetail()
aMan.citizen_id
```

```
class Person:
  '''A simple example class'''
  name = "Peter"
  def showdetail(self):
    self.citizen_id = 3701516086894
    print("name: {}".format(self.name))
    print("citizen ID: {}".format(self.citizen_id))
```

The operations understood by instance objects are attribute references
There are two kinds of valid attribute names:

- **Data attributes**
- **Methods**

Data attributes
- Class variables
- Instant variables

Class variables can be referred from class directly without instantiation

```
1  class Person:
2      '''A simple example class'''
3      name = "Peter"
4      def showdetail(self):
5          self.citizen_id = 3701516086894
6          print("name: {}".format(self.name))
7          print("citizen ID: {}".format(self.citizen_id))
```

A **method** is <u>a function</u> that "belongs to" an object.

Initialization of a Class Instance

we use a special method called __init__( )

Double underscore

def __init__(self):
self.data = [ ]

```
class Person:
    '''A simple example class with initialization'''
    name = "Peter"
    def __init__(self, name, id):
        self.name = name
        self.citizen_id = id

    def showdetail(self):
        print("name: {}".format(self.name))
        print("citizen ID: {}".format(self.citizen_id))
```

```
aMan = Person("Simon","1234554321")
```

Initialization of a Class's Instance

Question:

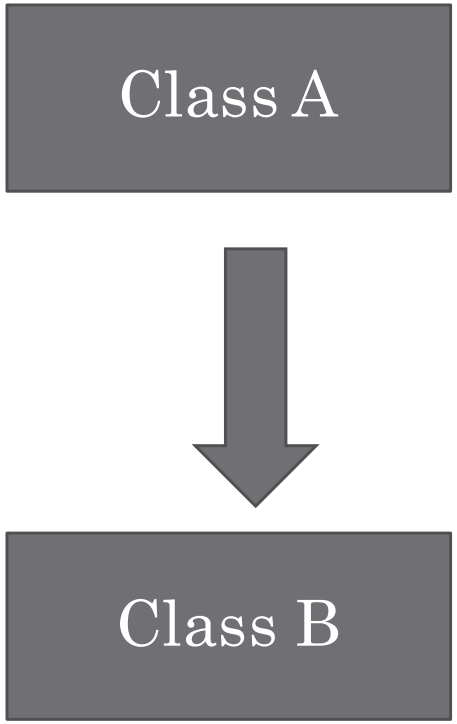
```
class Complex:
    def __init__(self,realpart,imgpart):
        self.r = realpart
        self.i = imgpart
```

```
>>> x = Complex(3.0,-4.5)
>>> x.r , x.i
```

answer

a) (3.0,-4.5)
b) 3.0,-4.5
c) realpart, imgpart
d) (realpart,imgpart)

Inheritance



Base Class

Derived Class

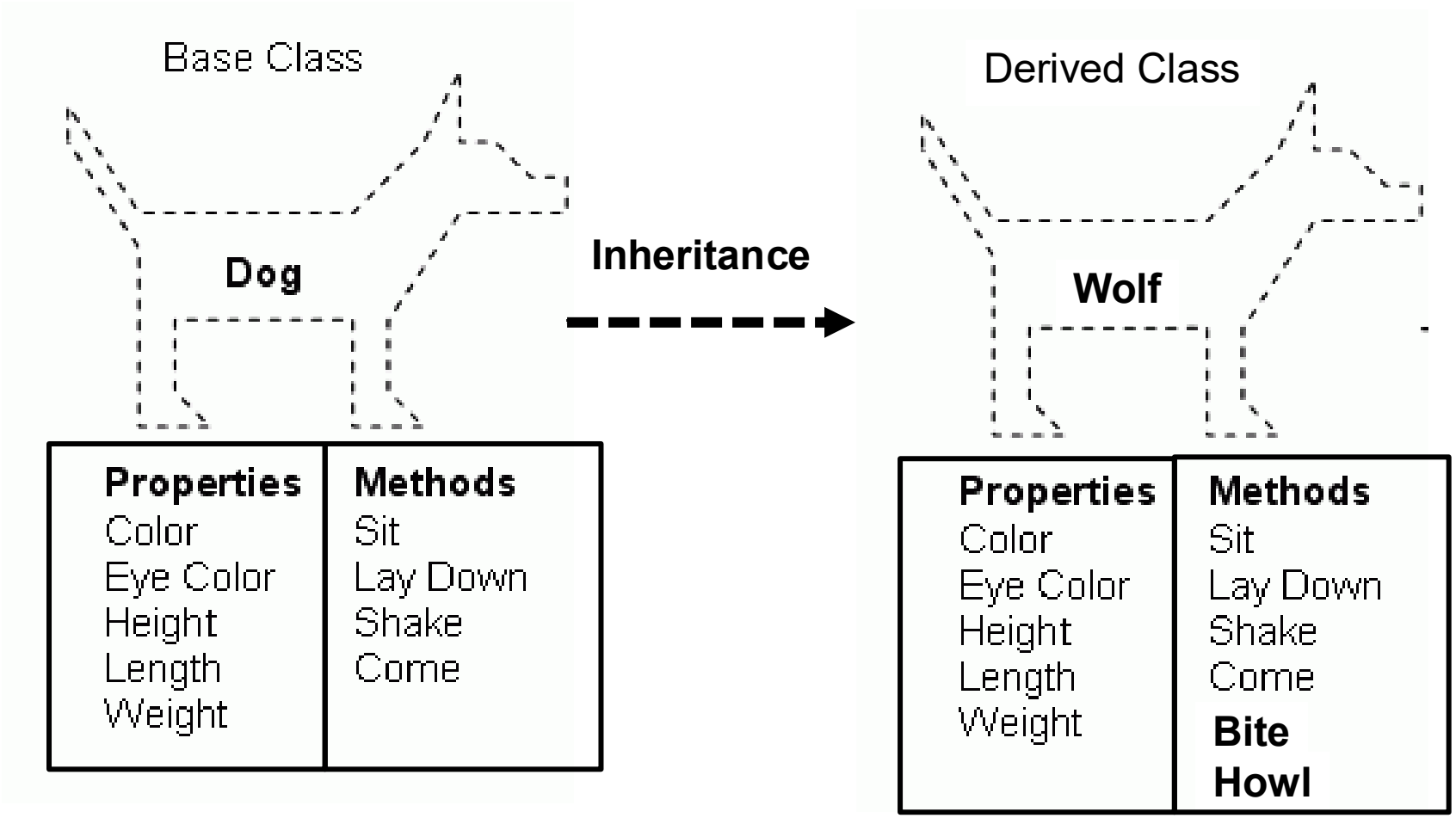
```
class DerivedClassName(BaseClassName):
    <statement-1>
    ...
    <statement-N>
```

overloading vs overriding

- overloading = same name but different parameters
- overriding = same name and same parameters

O v e r r i d i n g

Derived classes may override methods of their base classes.



Python has two built-in functions that work with inheritance:

- **isinstance()** check an instance's type: **isinstance(obj, type)**
- **issubclass()** check class inheritance: **issubclass(obj, subclass)**

Inheritance

```
class Person:
    '''A simple example class with initialization'''
    name = "Peter"
    def __init__(self, name, id):
        self.name = name
        self.citizen_id = id

    def showdetail(self):
        print("name: {}".format(self.name))
        print("citizen ID: {}".format(self.citizen_id))
```

```
class Student(Person):
  '''This is a simple example subclass'''
```

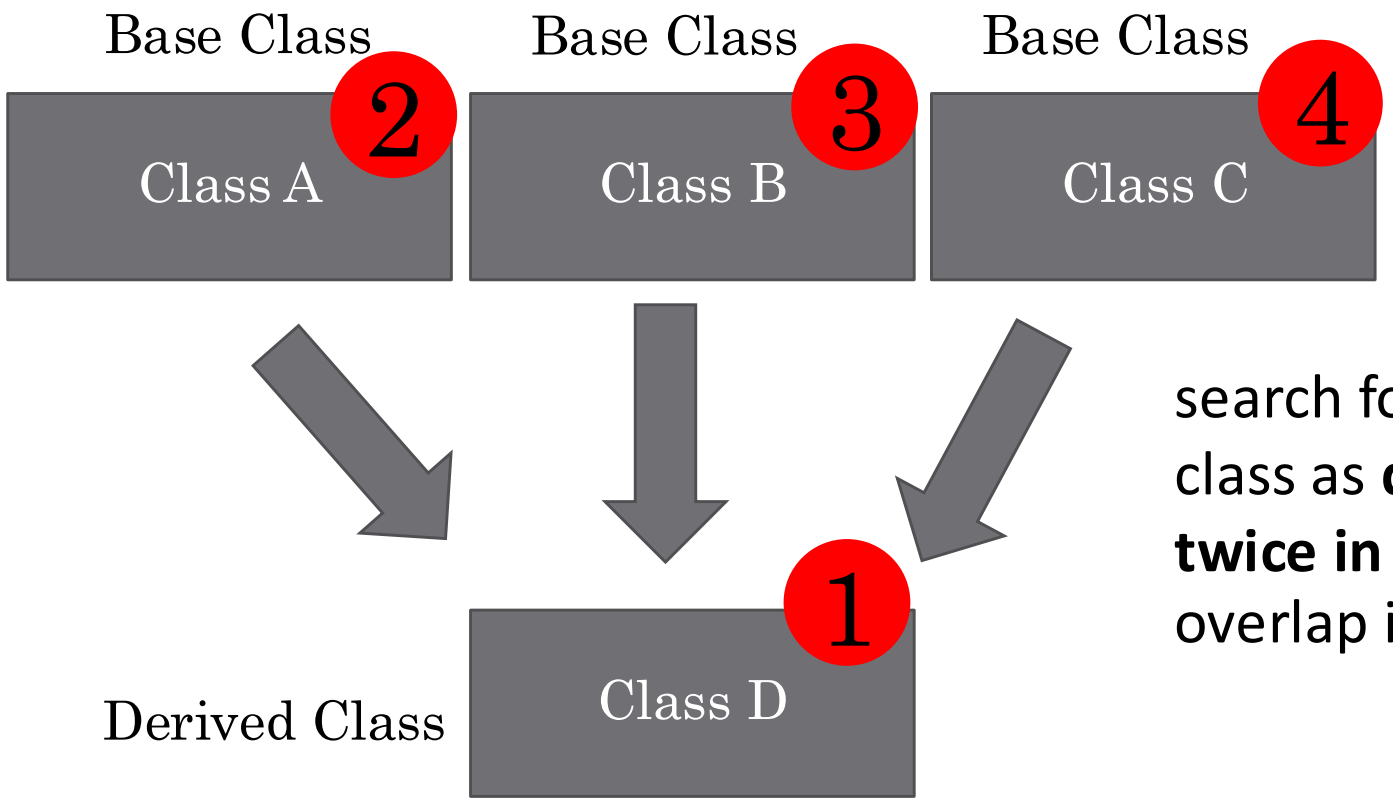
```
class Student(Person):
    '''This is a simple example subclass'''
    def __init__(self, name, id, age, school,level):
        self.name = name
        self.citizen_id = id
        self.age = age
        self.school = school
        self.level = level
    def showdetail(self):
        print("name: {:<25}".format(self.name))
        print("citizen ID: {}".format("----------"))
    def show_education_info(self):
        print("age: {:>5} years old".format(self.age))
        print("School name: {}".format(self.school))
        print("Level: {:>8}".format(self.level))
```

Overload

Override

add new method

Multiple Inheritance



search for attributes inherited from a parent class as **depth-first**, **left-to-right**, **not searching twice in the same class** where there is an overlap in the hierarchy.

```
class DerivedClassName(Base1, Base2, Base3):
    <statement-1>
    ...
    <statement-N>
```

Access modifier: Public, Protected, Private

- **Public variables /methods** are accessible from outside the class
- By default, Python define instance variables and methods as **Public**

```
class Person:
    '''A simple example class with initialization'''
    def __init__(self, name, id, age):
        self.name = name
        self.citizen_id = id
        self.age = age
    def showdetail(self):
        print("name: {:<25}".format(self.name))
        print("citizen ID: {}".format(self.citizen_id))
        print("age: {:>5} years old".format(self.age))
```

- **Protected variables/methods** of a class are accessible from within the class and are also available to its sub-classes
- No other environment should be permitted access to it
- Enable specific resources of the parent class to be inherited by the child class

a name prefixed with an underscore

e.g. _spam, _data, _name

should be treated as a non-public part
(whether it is a function, a method or a data member).

Not Enforced: Python's protected variables are not enforced by the language itself.
They can still be accessed and modified directly, but it is generally considered bad practice.

Example:

```
class Person:
    '''A simple example class with initialization'''
    def __init__(self, name, id, age):
        self.name = name
        self.citizen_id = id
        self._age = age

    def showdetail(self):
        print("name: {:<25}".format(self.name))
        print("citizen ID: {}".format(self.citizen_id))
        print("age: {:>5} years old".format(self._age))
```

Protected variable

Not Enforced: Python's protected variables are not enforced by the language itself. They can still be accessed and modified directly, but it is generally considered bad practice.

```
class Person:
    '''A simple example class with initialization'''
    def __init__(self, name, id, age):
        self.name = name
        self.citizen_id = id
        self._age = age
    def showdetail(self):
        print("name: {:<25}".format(self.name))
        print("citizen ID: {}".format(self.citizen_id))
        print("age: {:>5} years old".format(self._age))
        return
    def introduce(self):
        self._greeting()
    def _greeting(self):
      raise NotImplementedError("Subclasses should implement this method")
```

Use privated method with inherited class

- Enable specific resources of the parent class to be inherited by the child class

```
class Teacher(Person):
  def _greeting(self):
    print("Hello my name is {}. I am a teacher.".format(self.name))
```

```
class Student(Person):
    def _greeting(self):
        print("Hi everyone, my name is {}.".format(self.name))
```

Private Variables/Methods

- The double underscore __ prefixed to a variable makes it private
- strong suggestion not to touch it from outside the class
- Any attempt to do so will result in an **AttributeError:**

Example:

```
class Person:
    '''A simple example class with initialization'''
    def __init__(self, name, id, age):
        self.name = name
        self.__citizen_id = id  # privated
        self._age = age

    def showdetail(self):
        print("name: {:<25}".format(self.name))
        print("citizen ID: {}".format(self.__citizen_id))
        print("age: {:>5} years old".format(self._age))
        return
    def introduce(self):
        self._greeting()
    def _greeting(self):
      raise NotImplementedError("Subclasses should implement this method")
```

Setter, Getter Method

Setter Method is method for set the values to data attribute

Example:

```
def setValue(self, value):
    self.__data = value
```

Getter Method is method for get the value from data attribute

Example:

```
def getValue(self):
    return self.__data
```

```
class Person:
    '''A simple example class with initialization'''
    def __init__(self, name, id, age):
        self.name = name
        self.__citizen_id = id
        self._age = age

    def showdetail(self):
        print("name: {:<25}".format(self.name))
        print("citizen ID: {}".format(self.__citizen_id))
        print("age: {:>5} years old".format(self._age))
        return
    def introduce(self):
        self._greeting()
    def _greeting(self):
        raise NotImplementedError("Subclasses should implement this method")
    def get_citizen_id(self):     # getter method
        return self.__citizen_id
    def set_citizen_id(self,id):  # setter method
        self.__citizen_id = id
```

Summary: Difference between _, __ and __xx__ in Python

- **One underline in the beginning (Single pre underscore)**

  Python doesn't have real protected methods, so one underline in the beginning of a method or attribute means you shouldn't access this method, because it's not part of the API.
  e.g. _spam, _name, _data

  - ***single pre underscore*** doesn't stop you from accessing the *single pre underscore* variable
  - But ***single pre underscore*** effects the names that are imported from the module

Summary: Difference between _, __ and __xx__ in Python

- **Two underlines in the beginning (Double pre underscore)**
  use for avoid the method to be overridden by a subclass.
  e.g. __methodA( ), __bite( )

Double Pre Underscores are used for the <u>name mangling</u>

Example: Two underlines in the beginning

Question:

```
class A(object):
    def __method(self):
        print('I am a method in A')

    def method(self):
        self.__method()

>>> a = A()
>>> a.method()
```

```
class B(A):
    def __method(self):
        print('I am a method in B')

>>> b = B()
>>> b.method()
```

I'm a method in A

answer:

a) I am a method in A

b) I am a method in B

Summary: Difference between _, __ and __xx__ in Python

- **Two underlines in the beginning and in the end**

When you see a method like __this__, the rule is simple:

"don't call it."

- __init__( )
- __len__( )
- __add__( )
- __sub__( )
- __repr__( )
- __str__( )
- .....
- .....

Why? Because it means it's a method python calls, not you.

http://igorsobreira.com/2010/09/16/difference-between-one-underline-and-two-underlines-in-python.html

Iterators

An iterator is an object that contains a countable number of values.

To create an object/class as an iterator you must implement the methods __iter__() and __next__() to your object.

like __init__() for iterator

An **iterator** is an object that implements **next**, which is expected to return the next element of the iterable object that returned it, and raise a **StopIteration** exception when no more elements are available

Example of iterable objects (not iterator): list, tuple, dictionary, str, file

```
for element in [1,2,3]:
    print(element)

for element in (1,2,3):
    print(element)

for key in {'one':1,'two':2}:
    print(key)

for char in "123":
    print(char)

for line in open("myfile.txt"):
    print(line, end='')
```

Behind the scenes

- The for statement calls iter() on the container object.
- The function returns an iterator object that defines the method __next__() which accesses elements in the container one at a time.
- When there are no more elements, __next__() raises a StopIteration exception which tells the for loop to terminate.

```
>>> s = 'abc'
>>> it = iter(s)
>>> it
<iterator object at 0x00A1DB50>
```

```
>>> next(it)
```

answer:

a) a
b) b
c) c
d) ‘a’
e) ‘b’
f) ‘c’

Generators

Generators is subset of Iterators

Generators are a simple and powerful tool for creating iterators.
They are written like regular functions but use the **yield** statement whenever they want to return data.

```
def reverse(data):
    for index in range(len(data)-1,-1,-1):
        yield data[index]
```

```
>>> data = [ 1,2,3,4,5]
>>> rdata = reverse(data)
>>> next(rdata)
```

answer:

a) 5
b) 4
c) 3
d) 2
e) 1